

Pipelined CPU

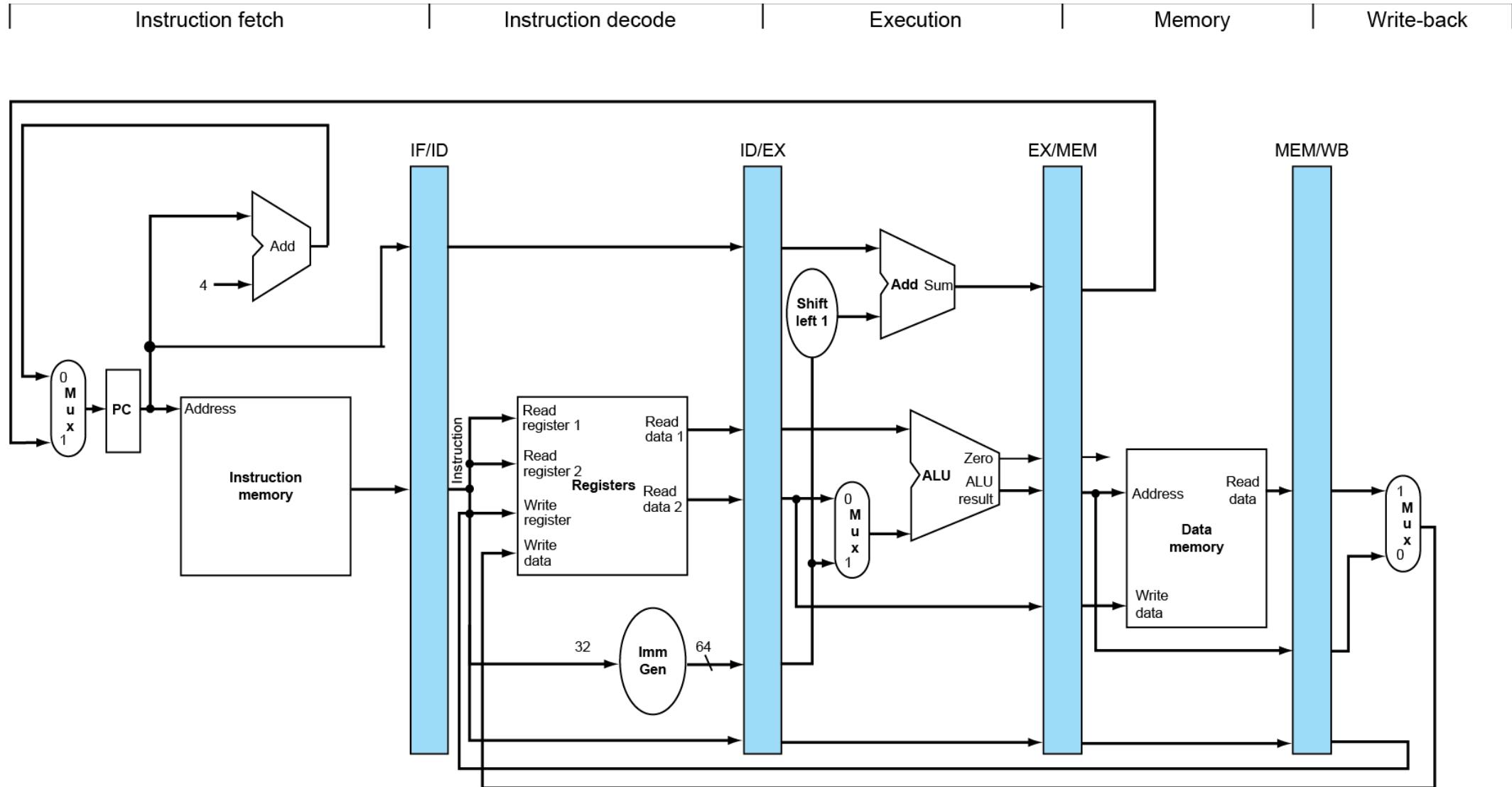
Jinyang Li

Based on the slides of Patterson and Hennessy

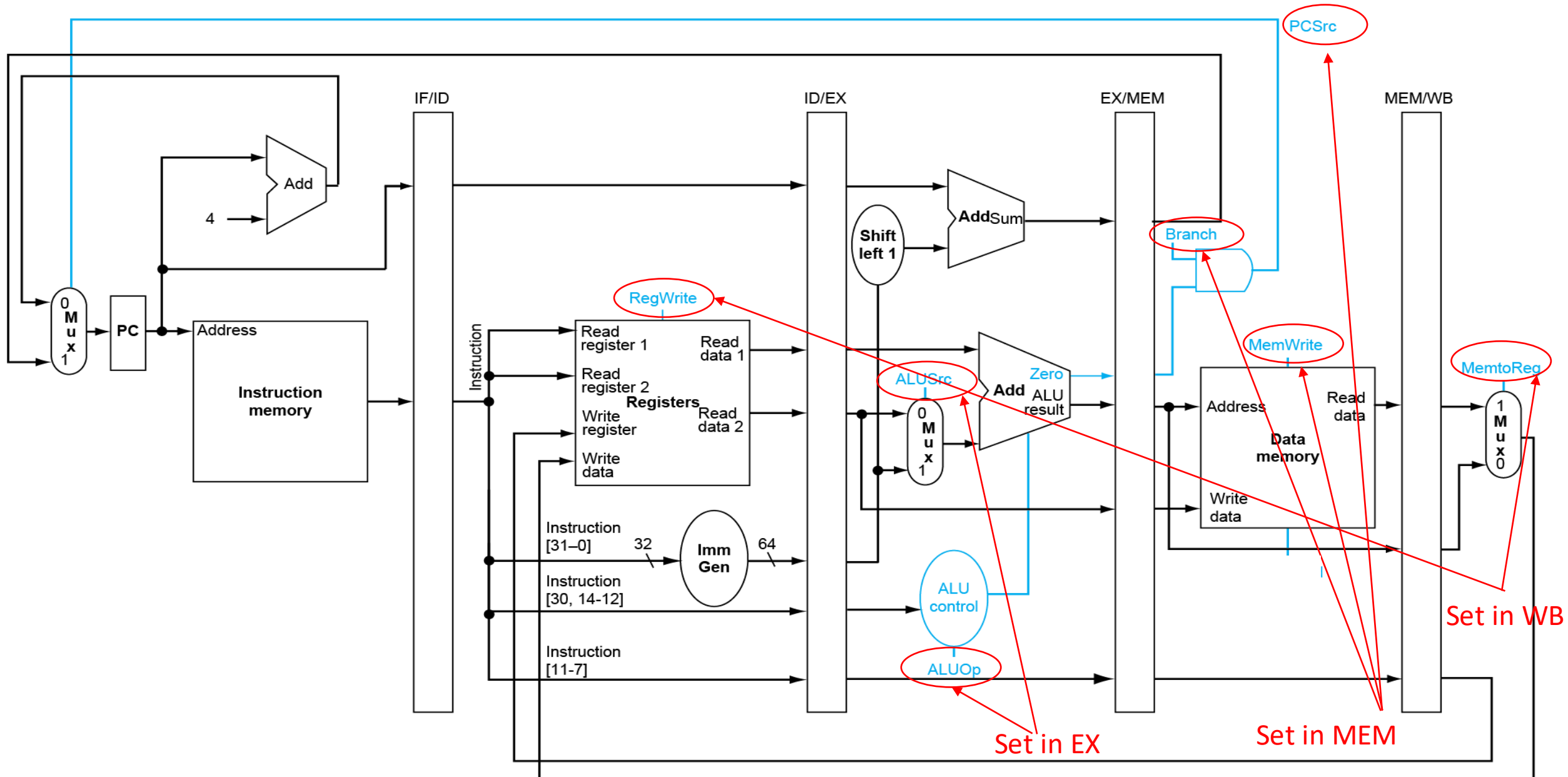
Today's lesson plan

- Pipeline control path
- Handling pipeline hazard
 - Structure
 - Data
 - Control

What we've learnt: basic 5-stage pipeline (datapath)

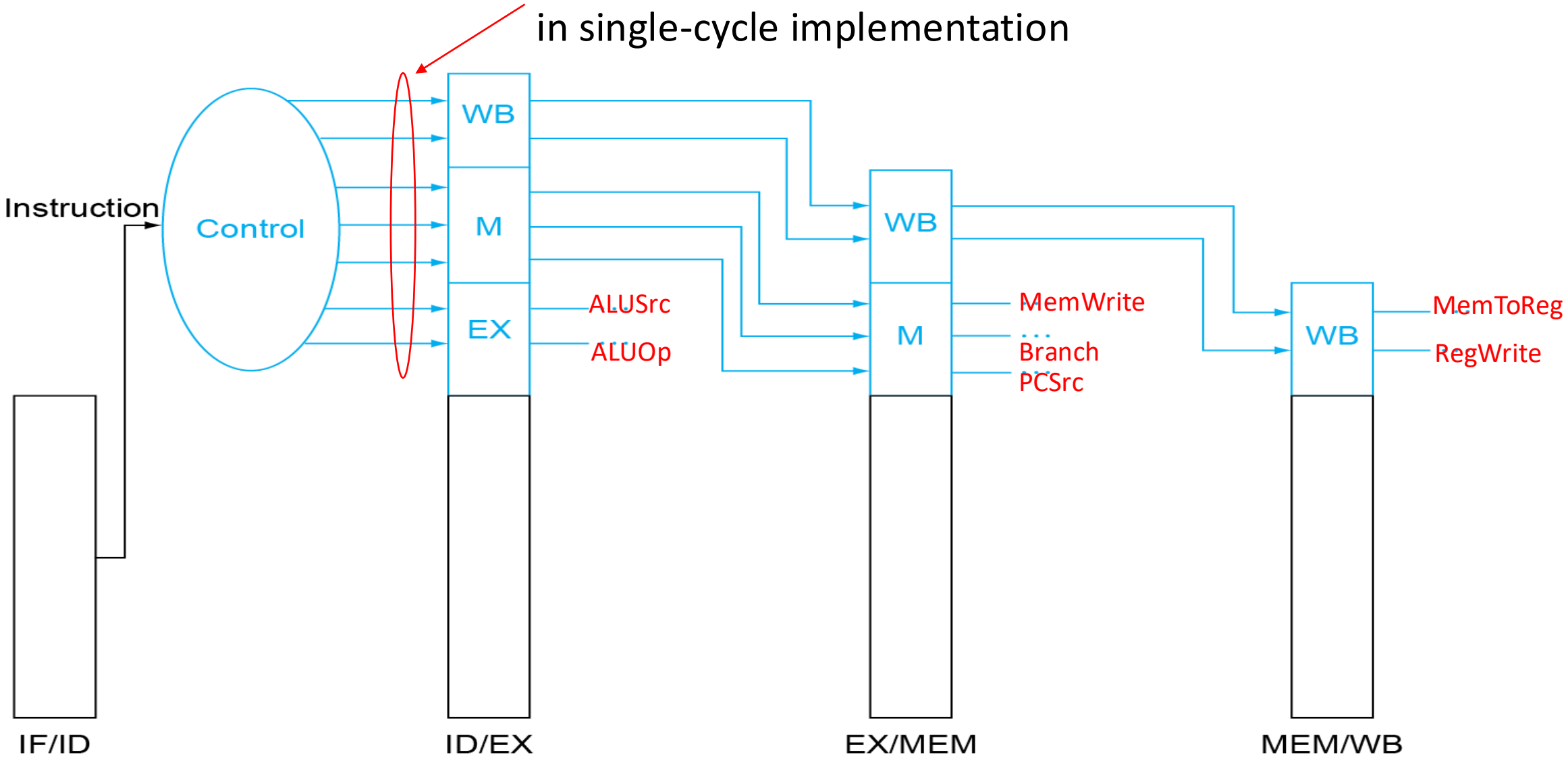


Pipelined Control (Simplified)



Pipelined Control

Control signals derived from instruction, same as in single-cycle implementation



Today's lesson plan

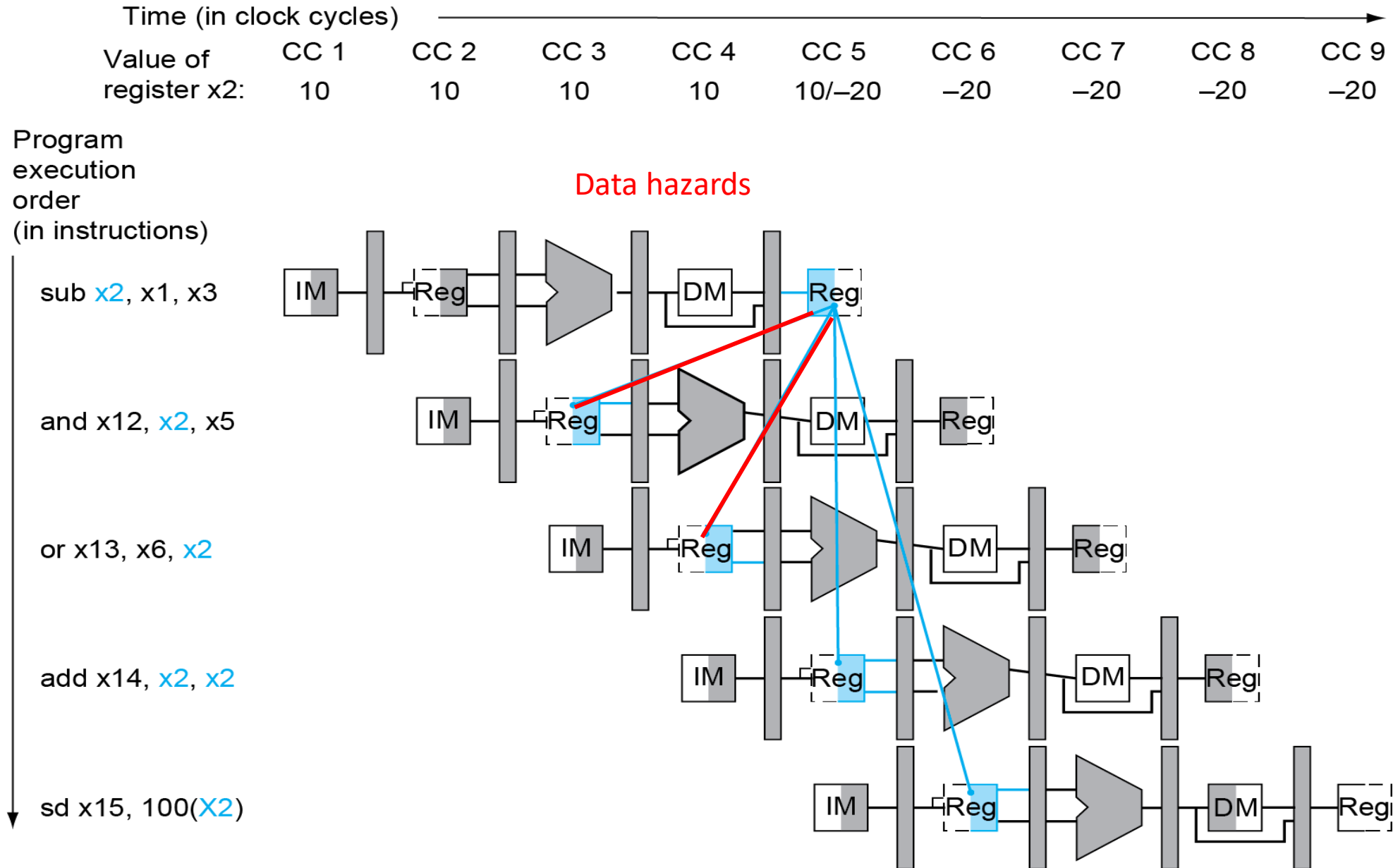
- Pipeline control path
- Handling pipeline hazard
 - Structure (solution: add resources)
 - Data (solution: forwarding or stall)
 - Control

Data Hazards in ALU Instructions

- Example instruction sequence:

```
sub  x2, x1, x3
and  x12, x2, x5
or   x13, x6, x2
add  x14, x2, x2
sd   x15, 100(x2)
```

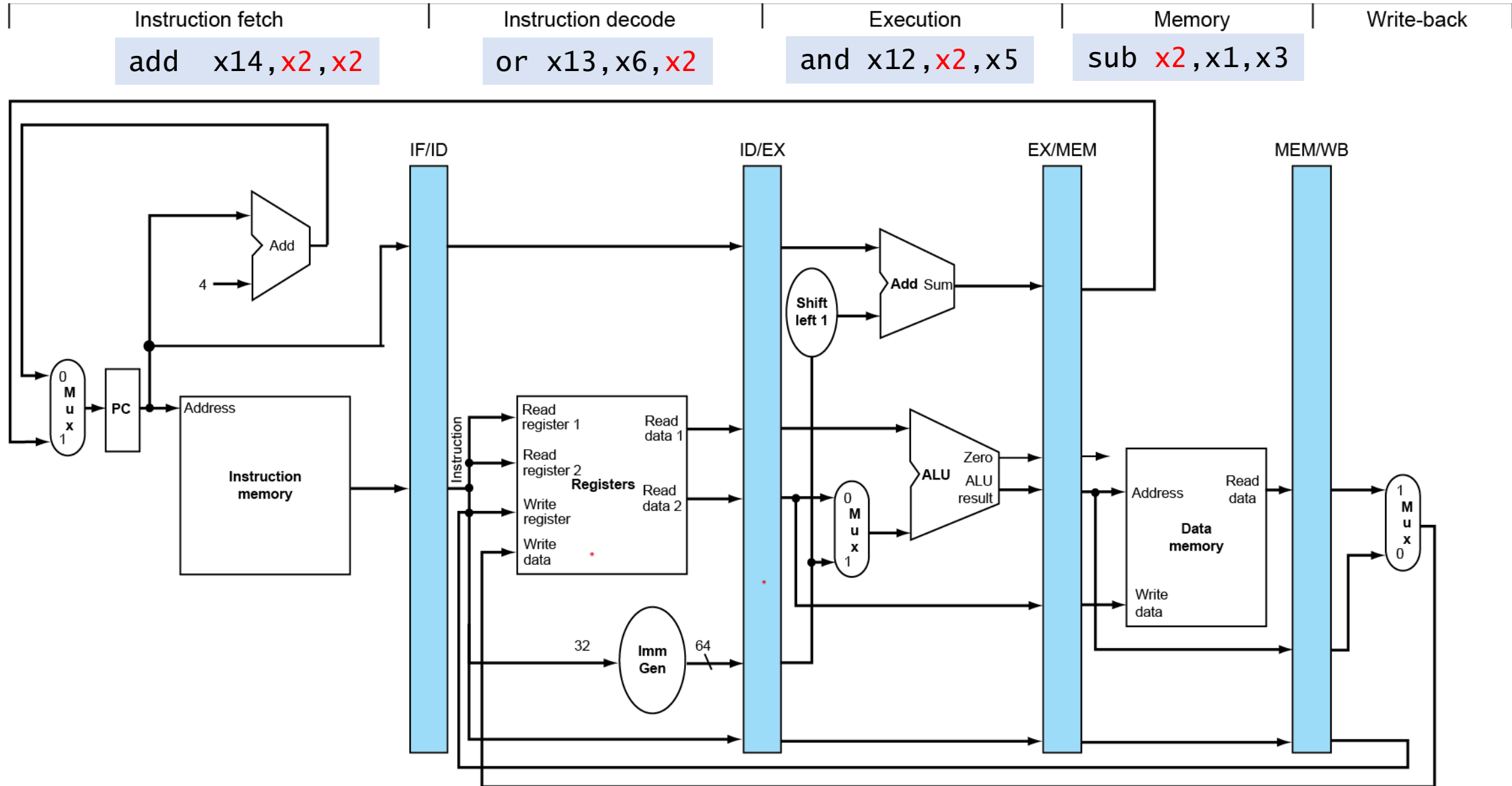
Data hazard in ALU instructions



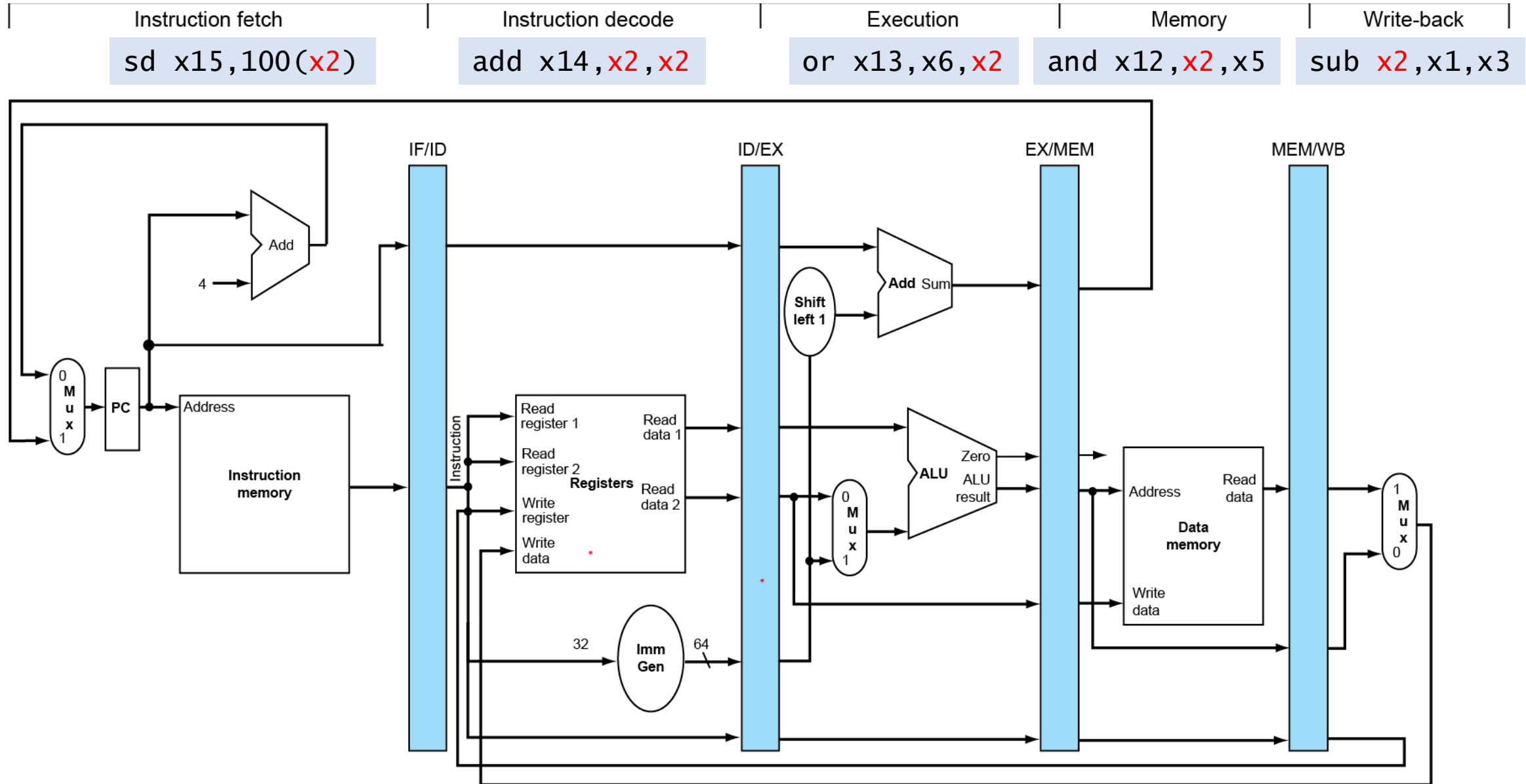
Data Hazards in ALU Instructions

- Solution: forwarding (aka bypassing)
 - Use the result after it's computed
 - Aka, don't wait for the result to be stored in register

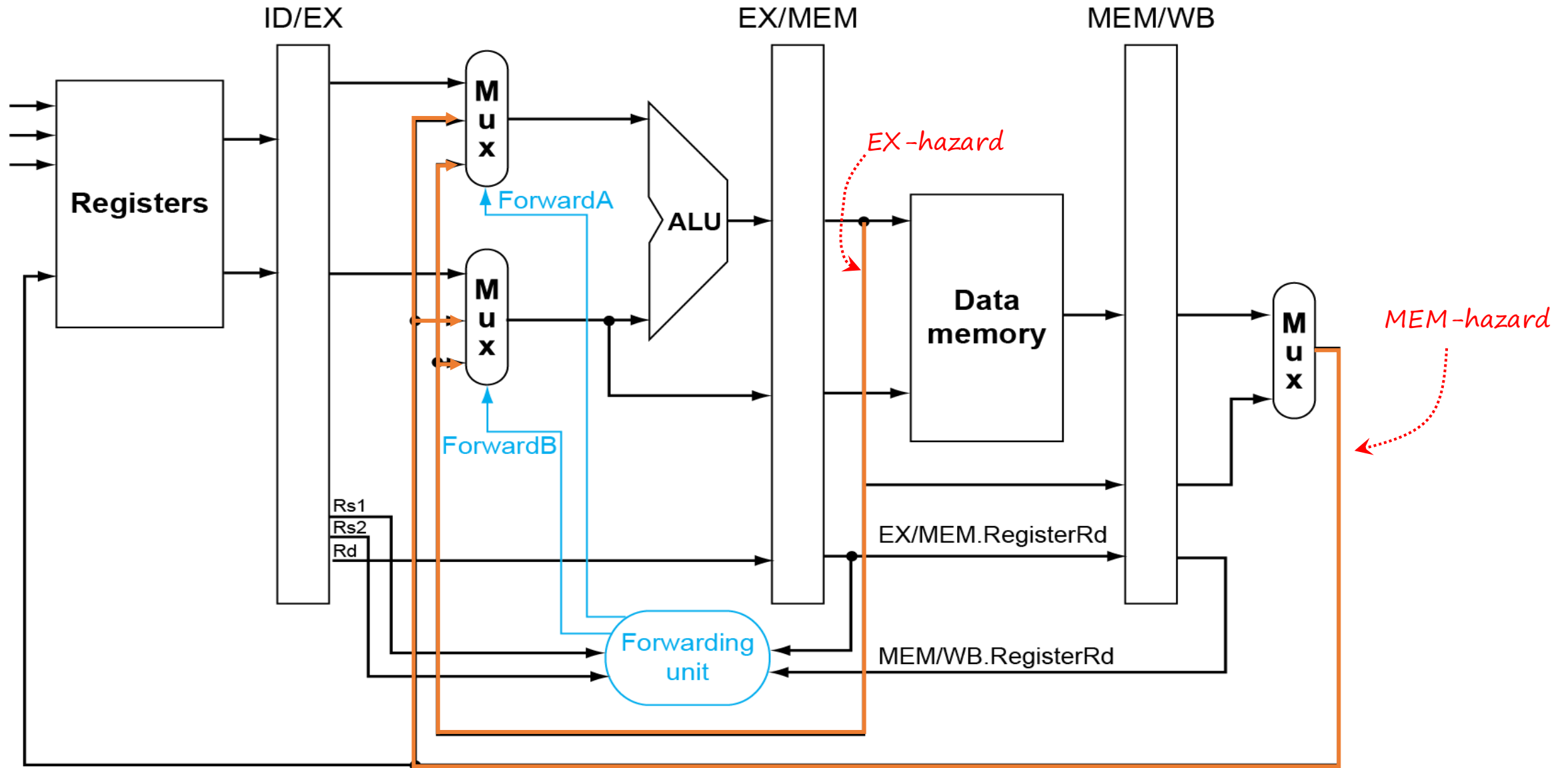
Using forwarding to resolve data hazard (cycle-4)



Using forwarding to resolve data hazard (cycle-5)



Forwarding Paths



Detecting when to use forwarded data

- Pass register numbers along pipeline
 - e.g., ALU operand register numbers (in EX stage) are:
 - ID/EX.RegisterRs1, ID/EX.RegisterRs2

- Data hazards when

1a. EX/MEM.RegisterRd = ID/EX.RegisterRs1

1b. EX/MEM.RegisterRd = ID/EX.RegisterRs2

2a. MEM/WB.RegisterRd = ID/EX.RegisterRs1

2b. MEM/WB.RegisterRd = ID/EX.RegisterRs2

EX-hazard:
Fwd from EX/MEM
pipeline reg

Mem-hazard: Fwd from
MEM/WB
pipeline reg

Detecting when to use forwarded data

- If forwarding instruction will write to a register!
 - EX/MEM.RegWrite, MEM/WB.RegWrite

AND

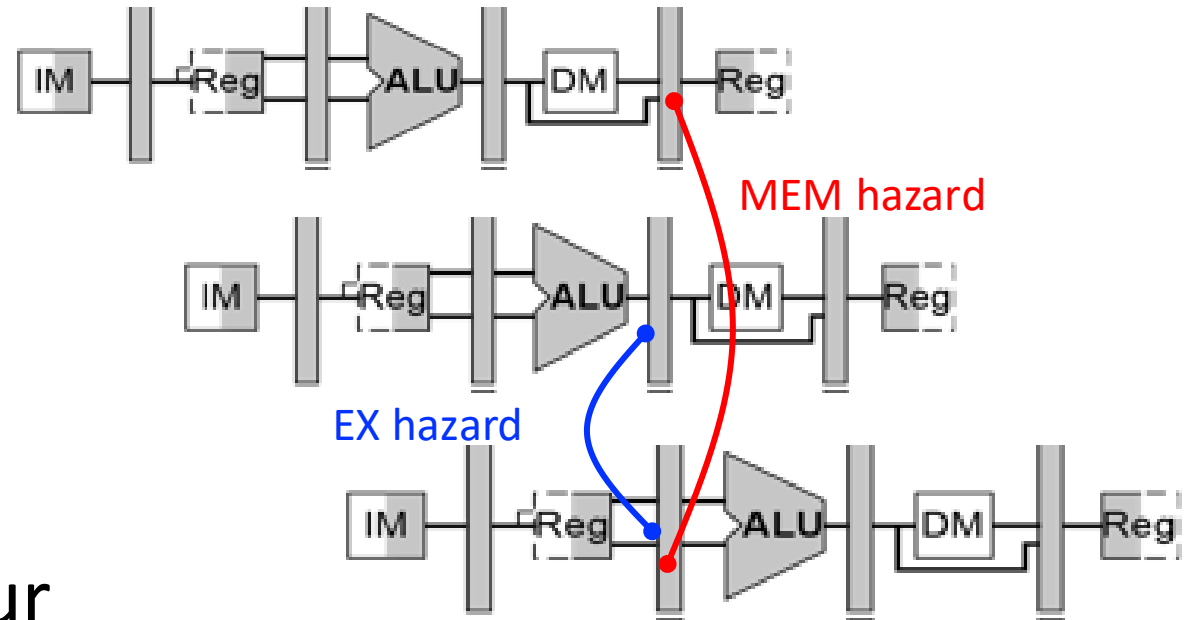
- if destination register (Rd) for that instruction is not x0
 - EX/MEM.RegisterRd $\neq$ 0,
MEM/WB.RegisterRd $\neq$ 0

x0 is always 0 in RISC-V



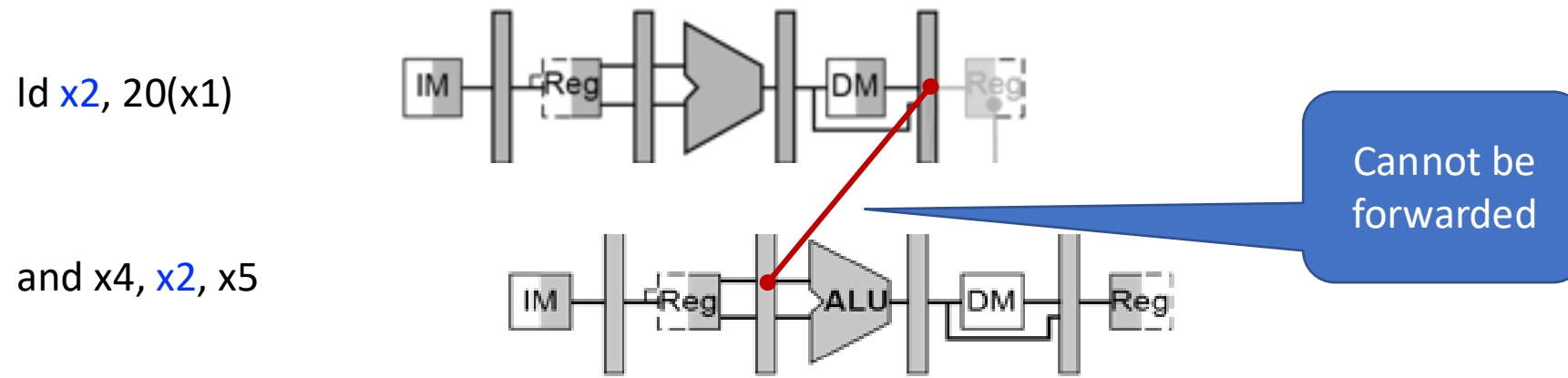
Double Data Hazard

- Consider the sequence:
 - add **x1**, x1, x2
 - add **x1**, **x1**, x3
 - add x1, **x1**, x4
- Both EX- and Mem-hazards occur
 - use the most recent
 - aka, only fwd Mem-hazard if EX hazard condition isn't true



Load-Use Data hazard Detection

- Load-use hazard cannot be resolved using forwarding alone

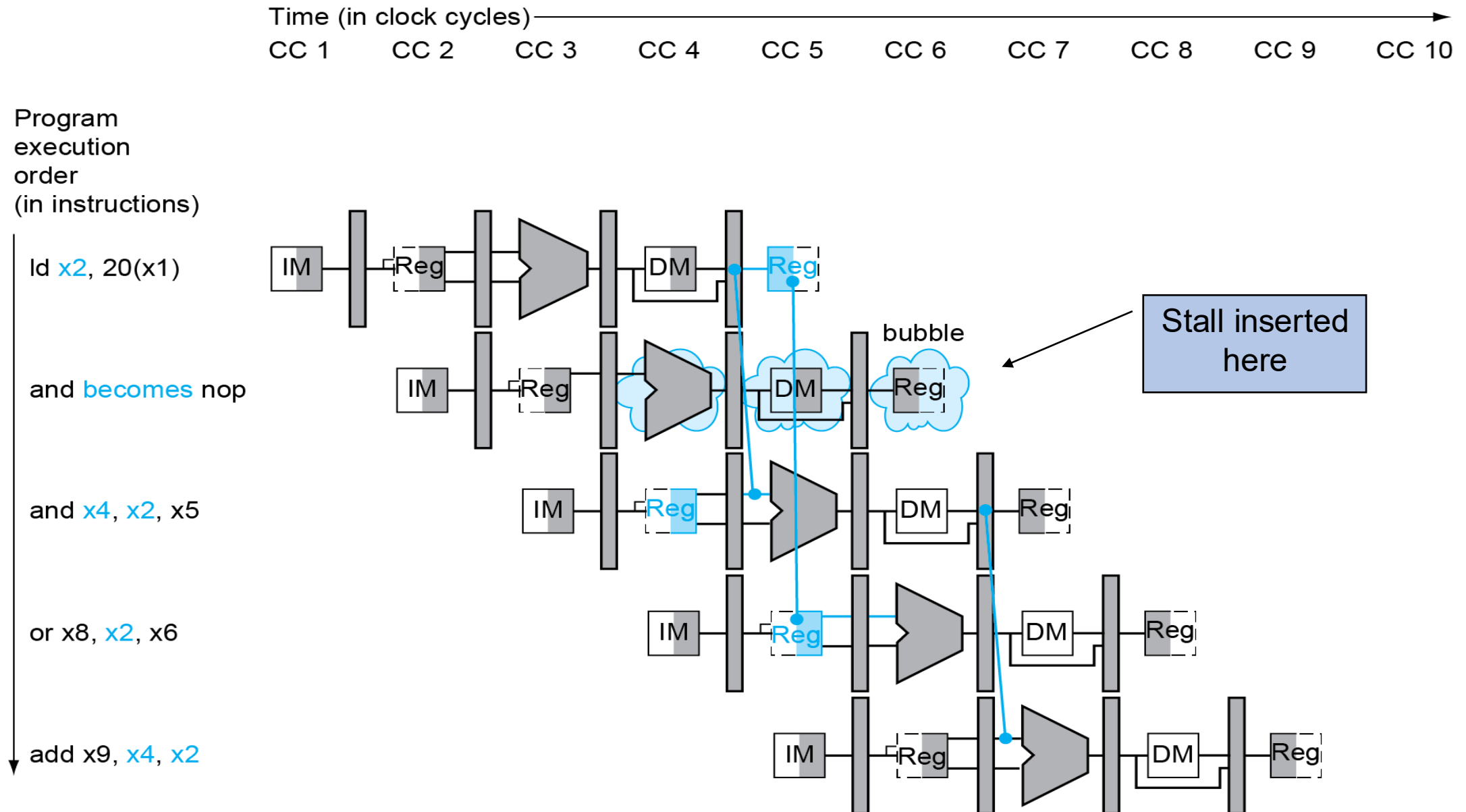


- Check load-use hazard using condition
 - $ID/EX.MemRead$ and $((ID/EX.RegisterRd = IF/ID.RegisterRs1) \text{ or } (ID/EX.RegisterRd = IF/ID.RegisterRs1))$
- If detected, stall and insert bubble

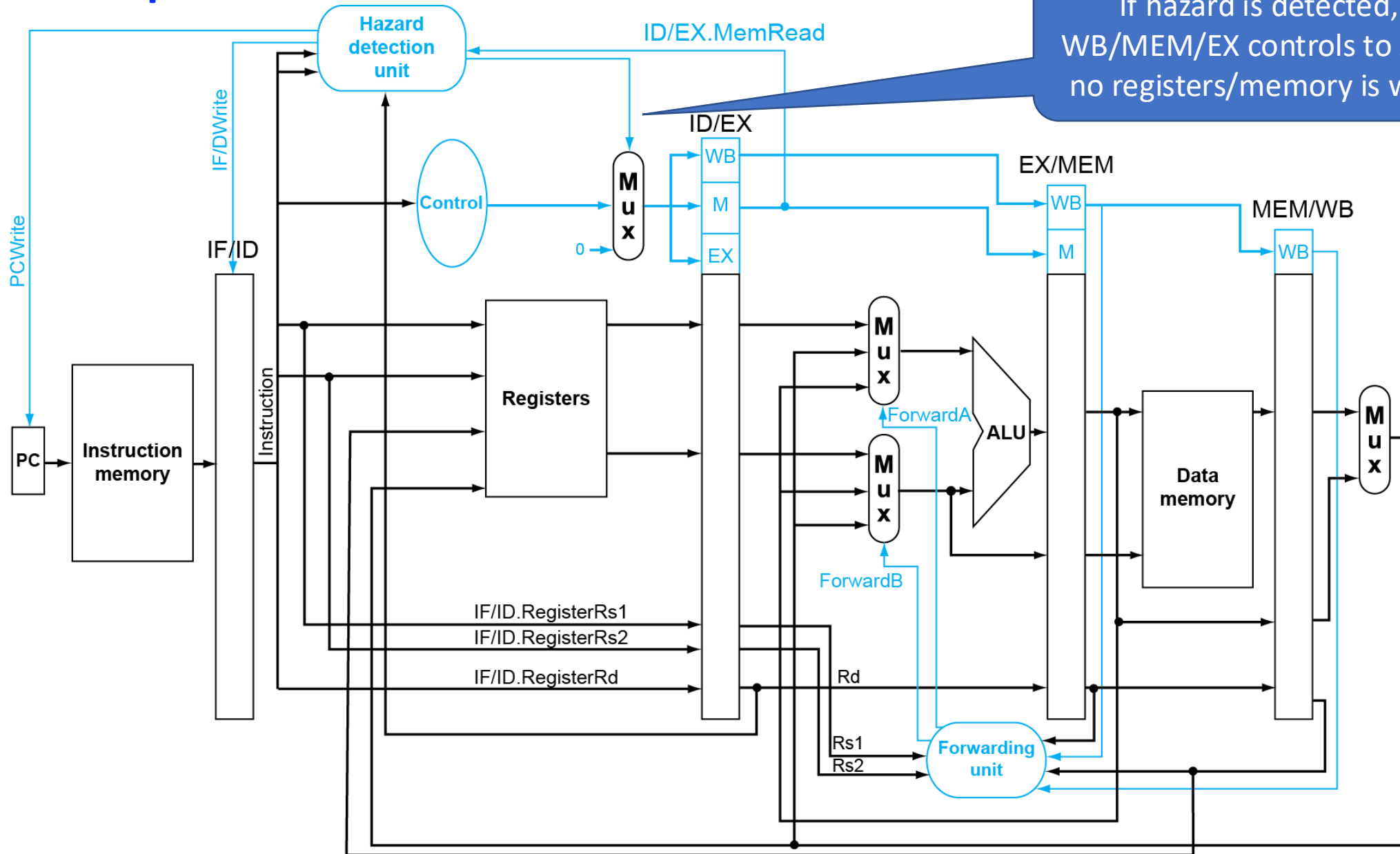
How to Stall the Pipeline (aka insert bubble)

- Force control values in ID/EX register to 0
 - EX, MEM and WB do nop (no-operation)
- Prevent update of PC and IF/ID register
 - Current instruction is decoded again
 - Following instruction is fetched again
 - 1-cycle stall allows MEM to read data for 1d
 - Can subsequently forward to EX stage

Load-Use Data Hazard



Datapath with Hazard Detection

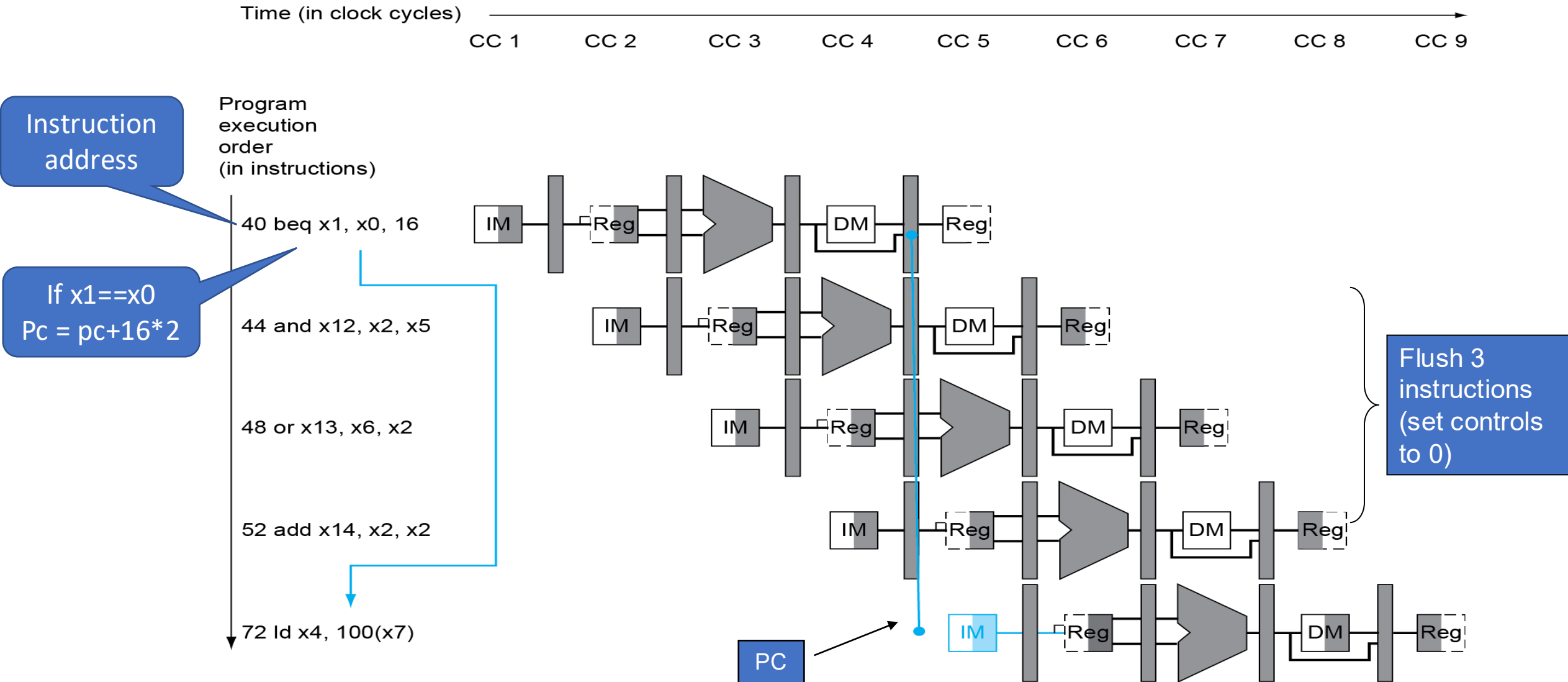


Today's lesson plan

- Pipeline control path
- Handling pipeline hazard
 - Structure (solution: add resources)
 - Data (solution: forwarding or stall)
 - Control (solution: stall, branch prediction)

Branch Hazard

- If branch outcome is determined in MEM

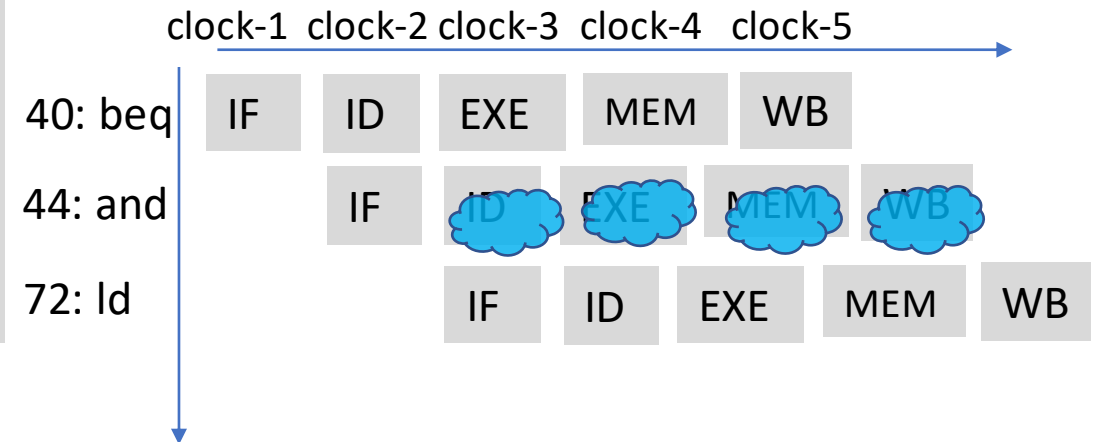


Reducing Branch Delay

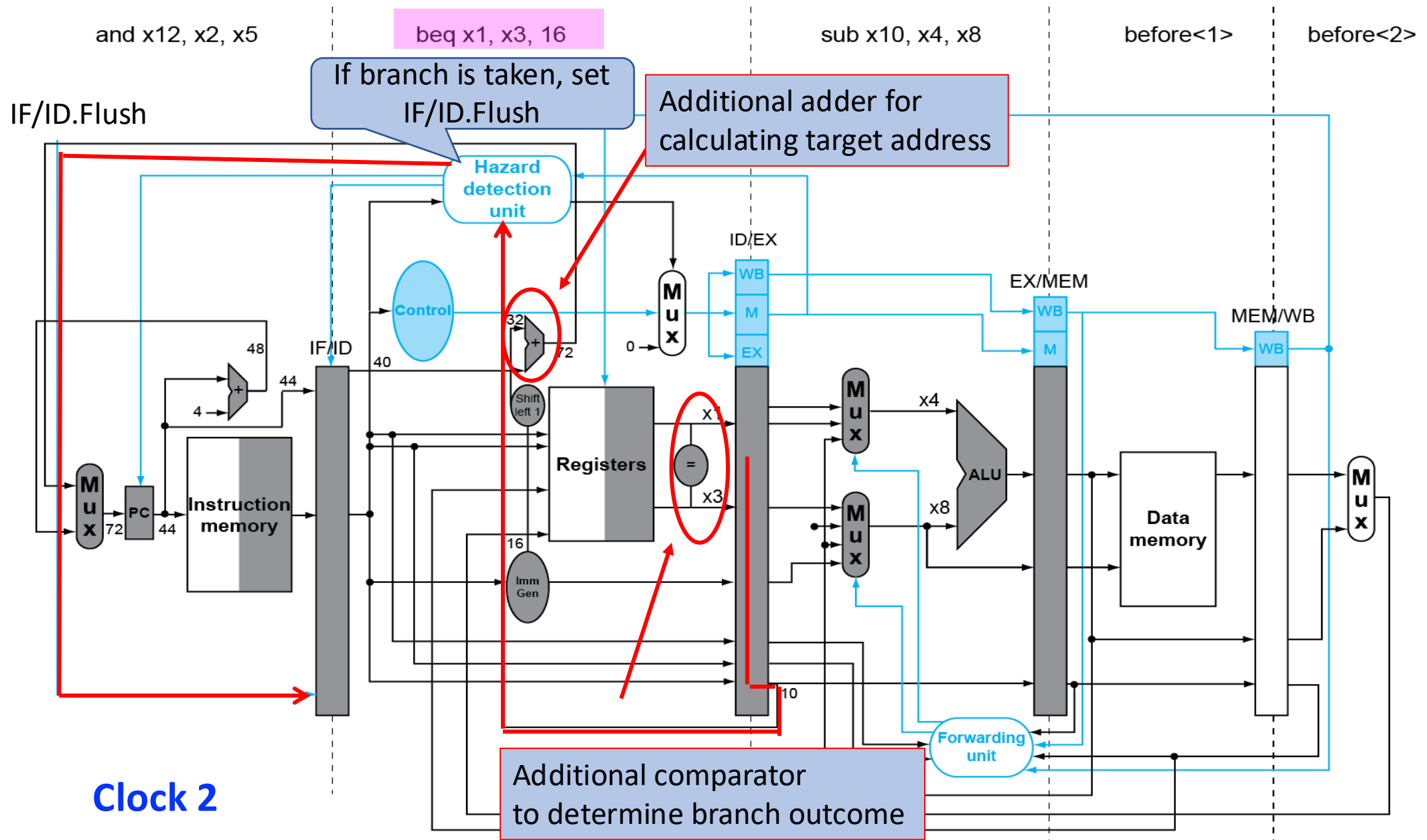
- Add hardware to determine branch outcome earlier (e.g. ID instead of MEM) → fewer instructions to flush

```
36:  sub  x10, x4, x8
40:  beq  x1,  x3, 16    // PC-relative branch
                          // to 40+16*2=72
44:  and  x12, x2, x5
48:  or   x13, x2, x6
52:  add  x14, x4, x2
56:  sub  x15, x6, x7
...
72:  ld   x4, 50(x7)
```

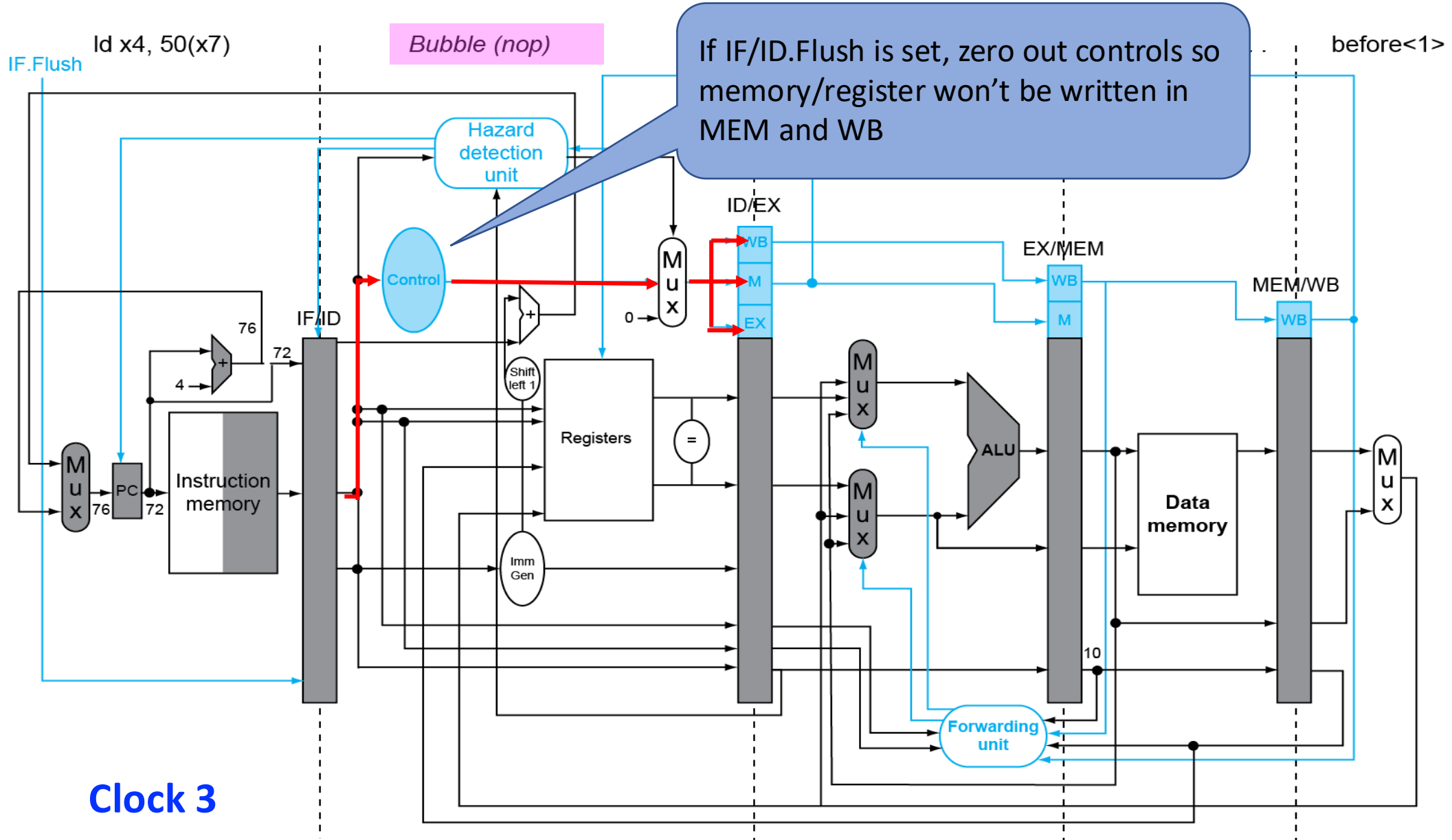
How many instructions to flush
if branch outcome is known in ID?



Branch determined in ID and is taken



Branch determined in ID and is taken



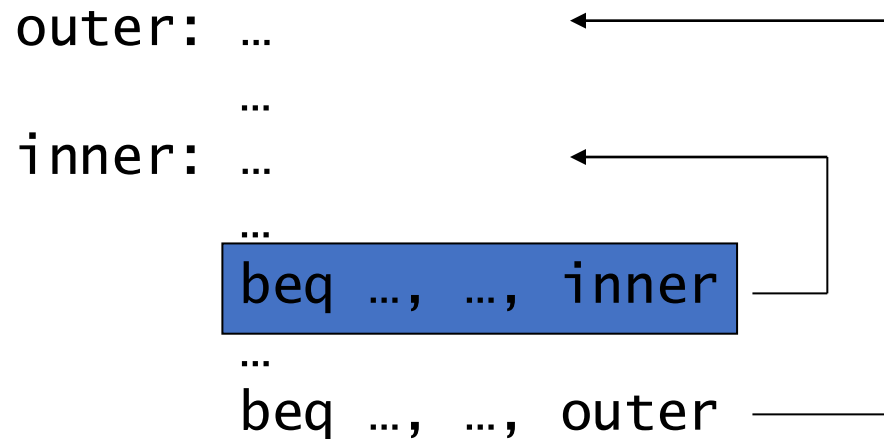
Dynamic Branch Prediction

- Our simple 5-stage pipeline's branch penalty is 1 bubble, but
 - In deeper pipelines, branch penalty is more significant
- Solution: dynamic prediction
 - Branch prediction buffer (aka branch history table)
 - Indexed by recent branch instruction addresses
 - Stores outcome (taken/not taken)
 - To execute a branch
 - Check table, expect the same outcome
 - Start fetching from fall-through or target
 - If wrong, flush pipeline and flip prediction

1-Bit Predictor: Shortcoming

- Inner loop branches mispredicted twice!

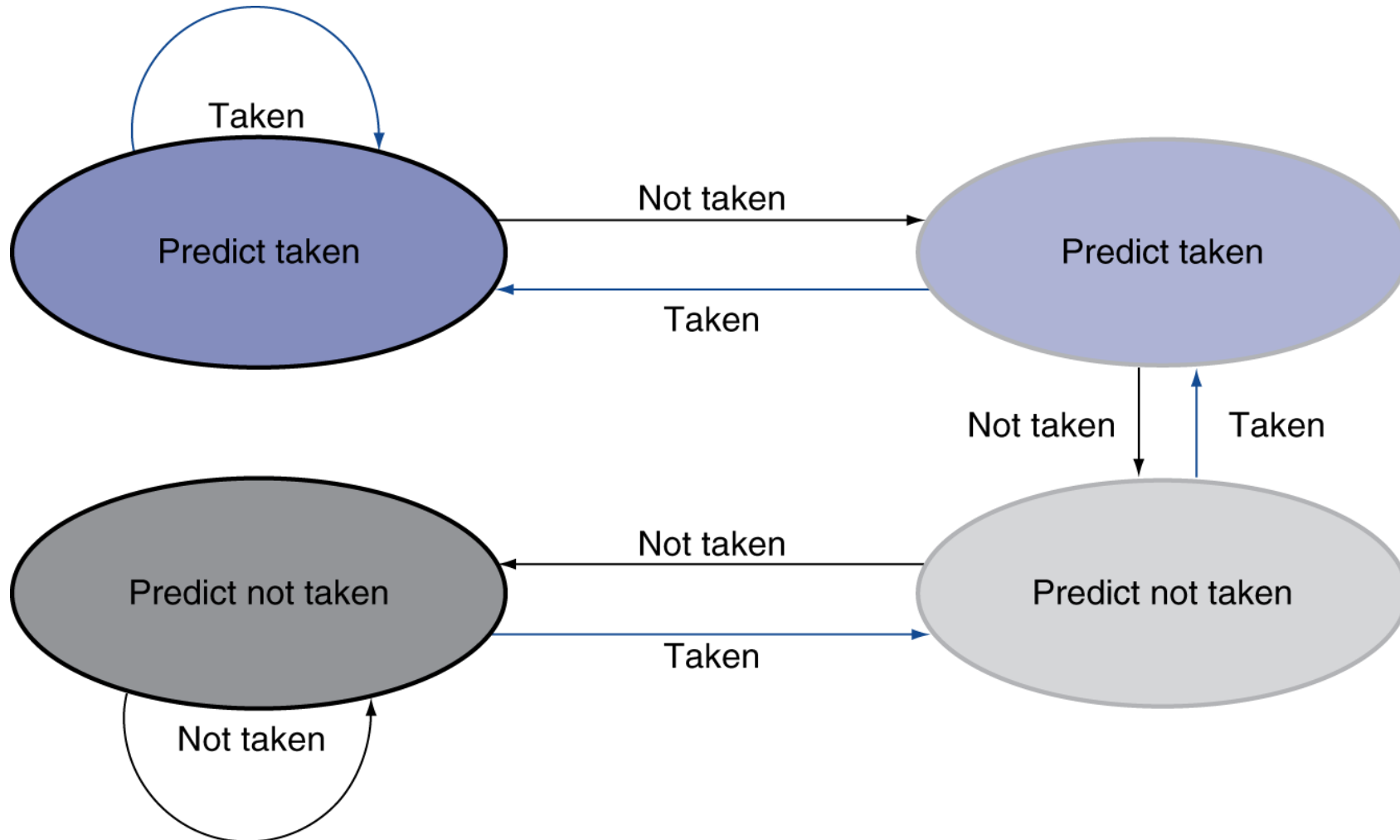
```
for (i = 0; i != 10; i++) {  
    for (j = 0; j != 100; j++) {  
        ...  
    }  
}
```



- Mispredicted as taken on last iteration of inner loop
- Then mispredicted as not taken on first iteration of inner loop next time around

2-Bit Predictor

- Only change prediction on two successive mispredictions



Calculating Branch Target (needed if branch is predicted taken)

- Even with predictor, still need to calculate the target address
 - 1-cycle penalty for a taken branch
- Branch target buffer
 - Cache of target addresses
 - Indexed by PC when instruction fetched
 - If hit and instruction is branch predicted taken, can fetch target immediately

Summary

- Pipeline increases throughput by overlapping execution of multiple instructions
- Pipeline hazard
 - Structure (solution: add resources)
 - Data (solution: forwarding, bubble)
 - Control (solution: bubble, branch prediction)